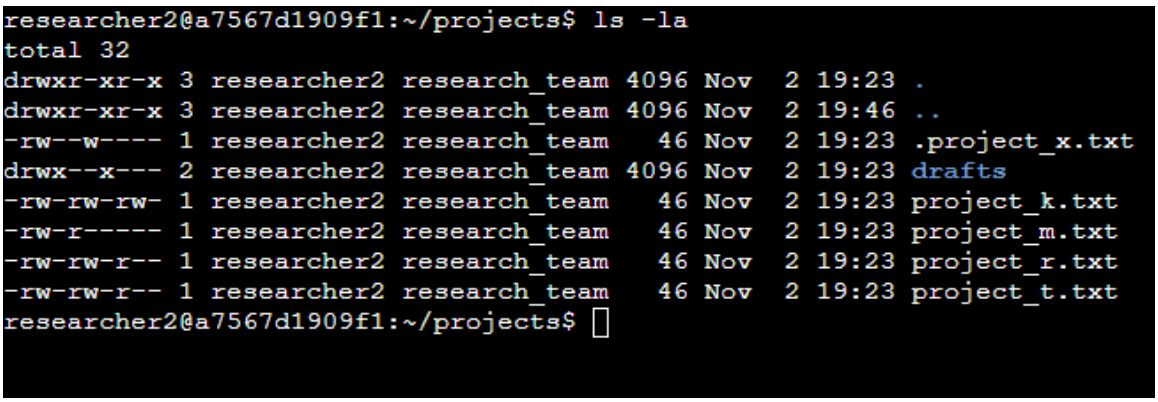
File permissions in Linux

Project description

In this Google Cybersecurity Certificate lab scenario, I acted as a security professional supporting a research team. Some files and one directory in the team's projects directory had permissions that did not match the access each user should have. Reviewing and correcting these permissions helps protect the system from unauthorized access. To complete this, I checked the current permissions and then corrected them on two files and one directory, as described below.

Check file and directory details

The following screenshot shows how I used a Linux command to check the current permissions in the projects directory:



The first line of the screenshot shows the command I entered, and the remaining lines show its output, a list of everything in the projects directory. I used ls with the -la option to get a detailed listing that also includes hidden files. The output shows one directory named drafts, one hidden file named .project_x.txt, and four other project files. The 10-character string in the first column shows the permissions set on each file or directory.

Describe the permissions string

Each 10-character string can be broken down to see who has access to an item and what they can do with it:

- **1st character:** The file type. A d means a directory, and a hyphen (-) means a regular file.
- **2nd–4th characters:** Read (r), write (w), and execute (x) permissions for the user. A hyphen (-) in any position means the user does not have that permission.
- **5th–7th characters:** Read (r), write (w), and execute (x) permissions for the group. A hyphen (-) in any position means the group does not have that permission.

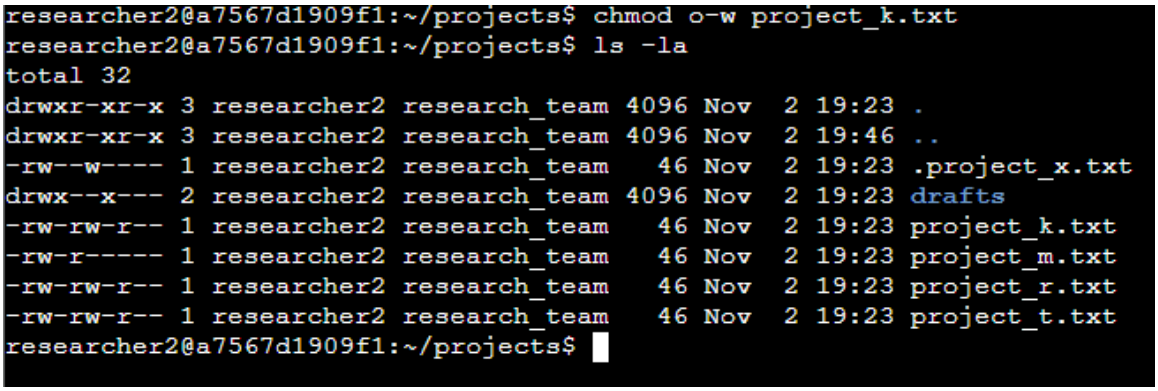
- **8th–10th characters:** Read (r), write (w), and execute (x) permissions for other, which means all other users on the system besides the user and the group. A hyphen (-) in any position means other does not have that permission.

For example, project_k.txt originally had the string -rw-rw-rw-. The first character is a hyphen, so project_k.txt is a regular file, not a directory. The 2nd, 5th, and 8th characters are all r, so the user, group, and other could all read it. The 3rd, 6th, and 9th characters are all w, so all three could also write to it. The 4th, 7th, and 10th characters are hyphens, so no one could execute it.

Change file permissions

In this scenario, the organization does not allow other to have write access to any file. Looking back at the permissions I listed, project_k.txt was the only file where other had write access, so I needed to remove it.

The following screenshot shows how I used Linux commands to make this change:

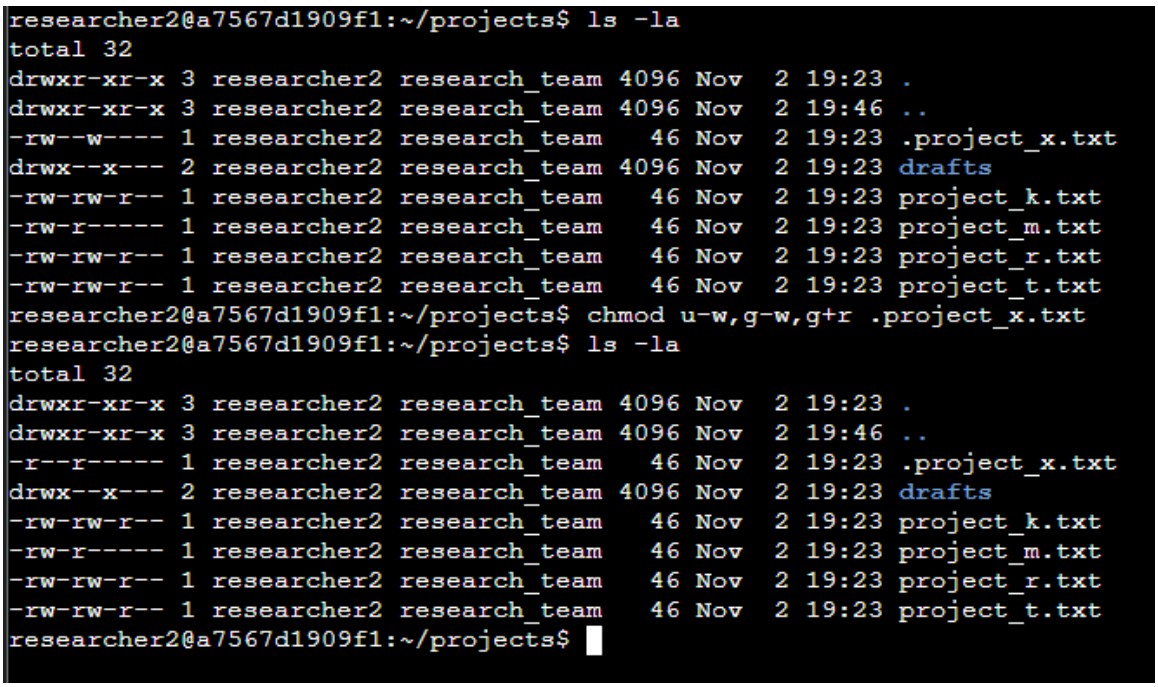


The first two lines of the screenshot show the commands I entered, and the rest of the lines show the output of the second command. The chmod command changes permissions on files and directories. Its first argument says which permissions to change, and its second argument names the file or directory. In o-w, o stands for other, and the minus sign removes the write (w) permission. Then I ran ls -la to confirm the change: project_k.txt now shows -rw-rw-r--.

Change file permissions on a hidden file

The research team archived .project_x.txt. No one should have write access to it, but the user and group should be able to read it.

The following screenshot shows how I used Linux commands to change its permissions:

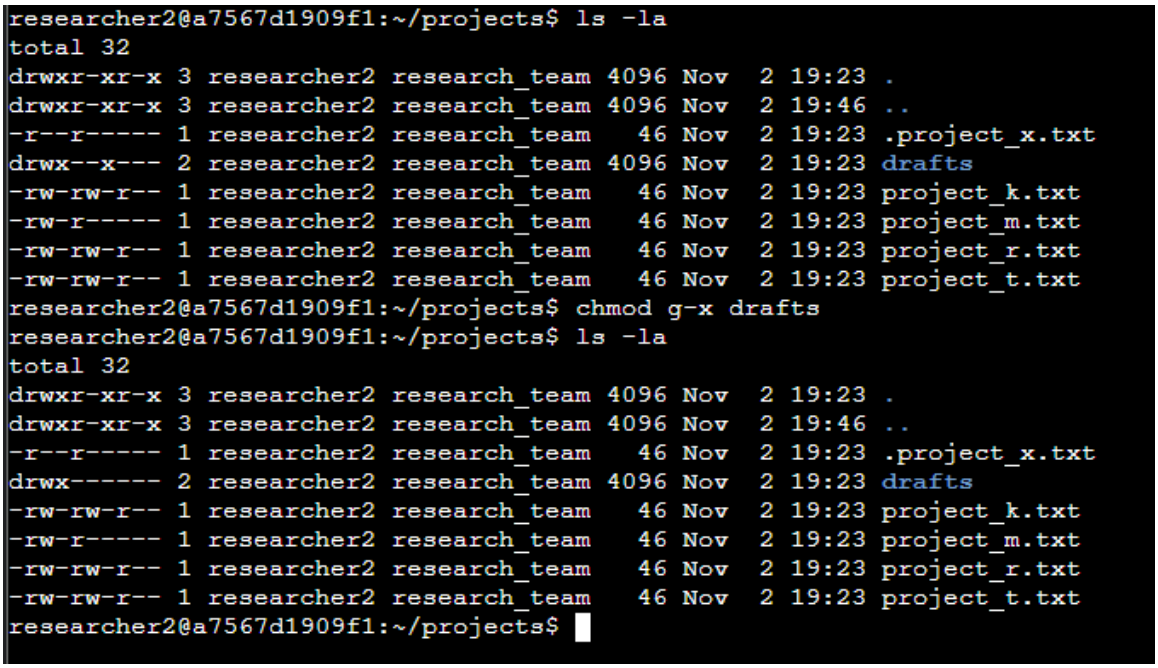


The screenshot shows three commands: ls -la to list the current permissions, the chmod command, and ls -la again to confirm the change. I can tell .project_x.txt is a hidden file because its name starts with a period (.), which is also why I need the -a option to see it. The first listing shows the file as -rw--w----. With u-w, I removed write access from the user. With g-w, I removed write access from the group, and with g+r, I gave the group read access. The result, -r--r-----, lets only the user and group read the file, and no one can write to it.

Change directory permissions

In this scenario, only the researcher2 user should have access to the drafts directory and its contents. For a directory, execute (x) permission is what lets a user enter it and access the files inside, so no one except researcher2 should have execute permission on it.

The following screenshot shows how I used Linux commands to change the permissions:



The first ls -la listing in the screenshot shows drafts as drwx--x---, which means the group had execute permission. I ran chmod g-x drafts to remove execute from the group, then ran ls -la again. Now drafts shows drwx------. The researcher2 user already had execute permission, so it did not need to be added, and other already had none.

Summary

In this lab scenario, I updated several permissions so the projects directory matched the scenario's authorization rules. I began with ls -la to review the current permissions, which guided each change I made. I then used chmod three times: to remove other's write access from project_k.txt, to make the archived .project_x.txt read-only for the user and group, and to limit the drafts directory to researcher2.